

Boletim de Produção Ambulatorial

Equipe 2 - *Clntilante*

O problema:

Quem é o stakeholder?

Stakeholder

Profissionais de saúde e faturista da Clínica de Especialidades Odontológicas (CEO/UFPE), que utilizam o processo de registro e geração do BPA no dia a dia.

Principal dor

1. Processo manual e burocrático;
2. Alto risco de erros e inconsistências;
3. Dificuldade na consolidação e envio de dados ao SUS.

Motivação

Automatizar o processo de registro e consolidação do BPA, **reduzindo erros, economizando tempo e aumentando a eficiência** da Clínica de Especialidades Odontológicas da UFPE.

O problema: Qual era a dor?

Stakeholder

Profissionais de saúde e faturistas da Clínica de Especialidades Odontológicas (CEO/UFPE), que utilizam o processo de registro e geração do BPA no dia a dia.

Principal dor

1. Processo manual e burocrático;
2. Alto risco de erros e inconsistências;
3. Dificuldade na consolidação e envio de dados ao SUS.

Motivação

Automatizar o processo de registro e consolidação do BPA, **reduzindo erros, economizando tempo e aumentando a eficiência** da Clínica de Especialidades Odontológicas da UFPE.

O problema: Por que resolver?

Stakeholder

Profissionais de saúde e faturistas da Clínica de Especialidades Odontológicas (CEO/UFPE), que utilizam o processo de registro e geração do BPA no dia a dia.

Principal dor

1. Processo manual e burocrático;
2. Alto risco de erros e inconsistências;
3. Dificuldade na consolidação e envio de dados ao SUS.

Motivação

Automatizar o processo de registro e consolidação do BPA, **reduzindo erros, economizando tempo e aumentando a eficiência** da Clínica de Especialidades Odontológicas da UFPE.

Visão geral

O BPA+ é uma solução web criada para modernizar o processo de geração do BPA na Clínica de Especialidades Odontológicas da UFPE. Por meio da centralização dos registros, da gestão de informações e da exportação automatizada de dados, a plataforma reduz a burocracia, minimiza erros e otimiza o trabalho da faturista.

Como
construímos?

Metodologia

2026

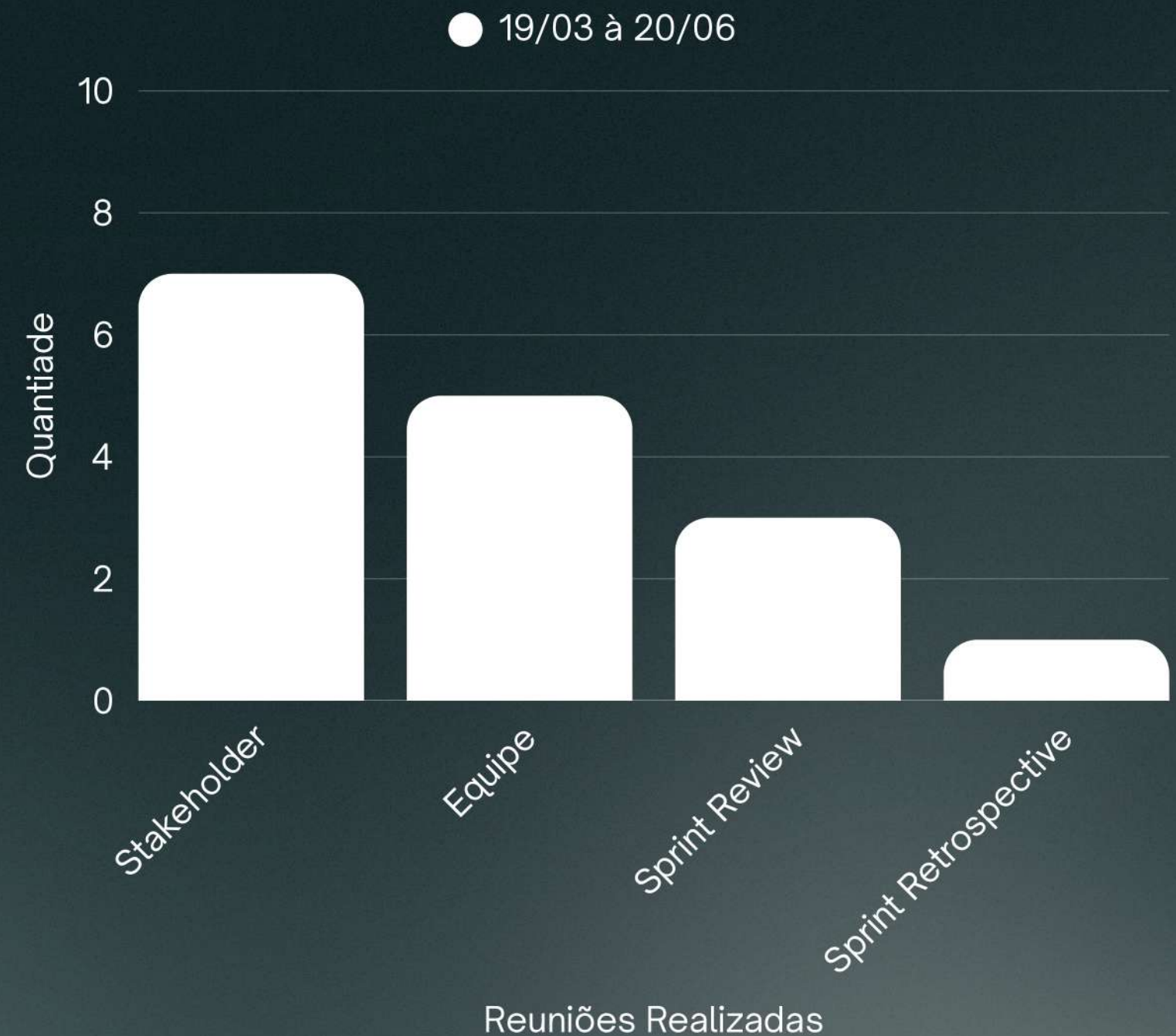
Metodologia ágil: Scrum

- Entregas contínuas por meio das Sprints;
- Acompanhamento eficiente com o stakeholder.

Cerimônias

- Foram realizadas 6 sprints após adaptação do calendário;
- Daily → Weekly: 2x semana, às seg e qui;
- Foram realizadas 3 sprints reviews e 1 sprint retrospective.

Reuniões Realizadas Durante o Desenvolvimento



Divisão de Responsabilidades

2026

Membro	Responsabilidade
Layse Gomes	Product Owner
Ana Clara Bizarria	Gerente de Projeto
Jaianny Souza	Desenvolvedora Backend
M ^a Cláudia Rocha	Desenvolvedora Backend
Samara Petrilly	Desenvolvedora Backend
Byanca Souza	Desenvolvedora Frontend
Raiana Donato	Desenvolvedora Frontend

Organização

2026



Jira & Confluence

Epic - 1 ou mais *User Stories*;
Sprint - 1 a 2 Epics.

Board utilizados no Jira:

Resumo, Orientações, Backlog e Cronograma.

Fluxo de desenvolvimento:

1. Padrões de semântica para criação de *branches* e envio de *commits*;
2. Para cada *feature* nova - 1 Pull request e 1 Code review;
3. A *feature* só é enviada a main com no mínimo um Code review realizado.

Organização



Google Drive & GitHub

Google Drive

- Concentração da documentação principal;
- Concentração do registro de reuniões;
- Concentração de documentação de apoio;
- Entregas de equipe durante o desenvolvimento.



GitHub

Versionamento do código fonte.

Requisitos & *User Stories*

Product Backlog & User Stories²⁰²⁶

Principais Épicos - *Epics*

- ⚡ SCRUM - 13 Registro de BPA
- ⚡ SCRUM - 17 Relatório BPA
- ⚡ SCRUM - 32 Documentação
- ⚡ SCRUM - 56 Sistema de Cadastros
- ⚡ SCRUM - 205 BUG

- Hierarquia de épicos;
- Cada épico - agrega valor ao desenvolvimento.

Product Backlog & User Stories²⁰²⁶

Principais Épicos - *Epics*

- ⚡ SCRUM - 13 Registro de BPA
- ⚡ SCRUM - 17 Relatório BPA
- ⚡ SCRUM - 32 Documentação
- ⚡ SCRUM - 56 Sistema de Cadastros
- ⚡ SCRUM - 205 BUG

User story de exemplo: Cadastro de Paciente

Como faturista,
quero cadastrar um paciente informando seus dados pessoais e de identificação,
para associar corretamente os atendimentos realizados ao paciente correspondente.

- De acordo com os critérios *INVEST*;
- 1 a 2 critérios de aceite para cada user story e 1 cenário de falha.

Rastreabilidade

- Foi realizada diretamente através das nossas *branches* do Git e da padronização nos nomes dos *commits*;
- 1 *feature* - 1 *branch*;
- Entendimento da evolução de uma *feature* apenas lendo a árvore de commits e as *branches*.

Arquitetura e Decisões arquiteturais

Arquitetura

2026

Camada	Decisão
Aplicação - Backend	Node.js + Express
Apresentação - Frontend	React via Vite
Persistência - Banco de Dados	PostgreSQL

Principais ADR's

ADR 002 - Adoção de Arquitetura Monolítica (Monolito)

- Alternativa: Ecossistema distribuído ou microsserviços;
- Justificativa principal: Garantir velocidade de desenvolvimento e reduzir a complexidade de rede.

ADR 004: Evolução da Arquitetura do Banco de Dados (Local para Nuvem)

- Alternativa: PostgreSQL rodando estritamente de forma local;
- Justificativa principal: Unificação do nosso ambiente de testes e garantiu que o sistema pudesse ser acessado de qualquer lugar.

ADR 009 - Centralização de Dependências com npm workspaces

- Alternativa: Manter pastas totalmente isoladas;
- Justificativa principal: Permite instalar tudo com um único comando na raiz do projeto.

Qualidade

Testes

2026

Unitários

- Vitest : 8 arquivos de teste com 62 testes aprovados

Figura 1. Evidência testes unitários do sistema de Boletim de Produção Ambulatorial.

```
Test Files  8 passed (8)
Tests      62 passed (62)
Start at   09:38:07
Duration   3.49s (transform 628ms, setup 0ms, import 1.93s, tests 151ms, environment 1ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Fonte: As autoras, 2026.

Integração (manual)

- Login de usuário
- Atualização de registros
- Recuperação de senha

E2E (manual)

- Fluxo completo do BPA

Refatoração

2026

Code smells: Código Duplicado - Duplicade Code

- CPF
- CNS
- verificação de duplicidade
- Outros campos

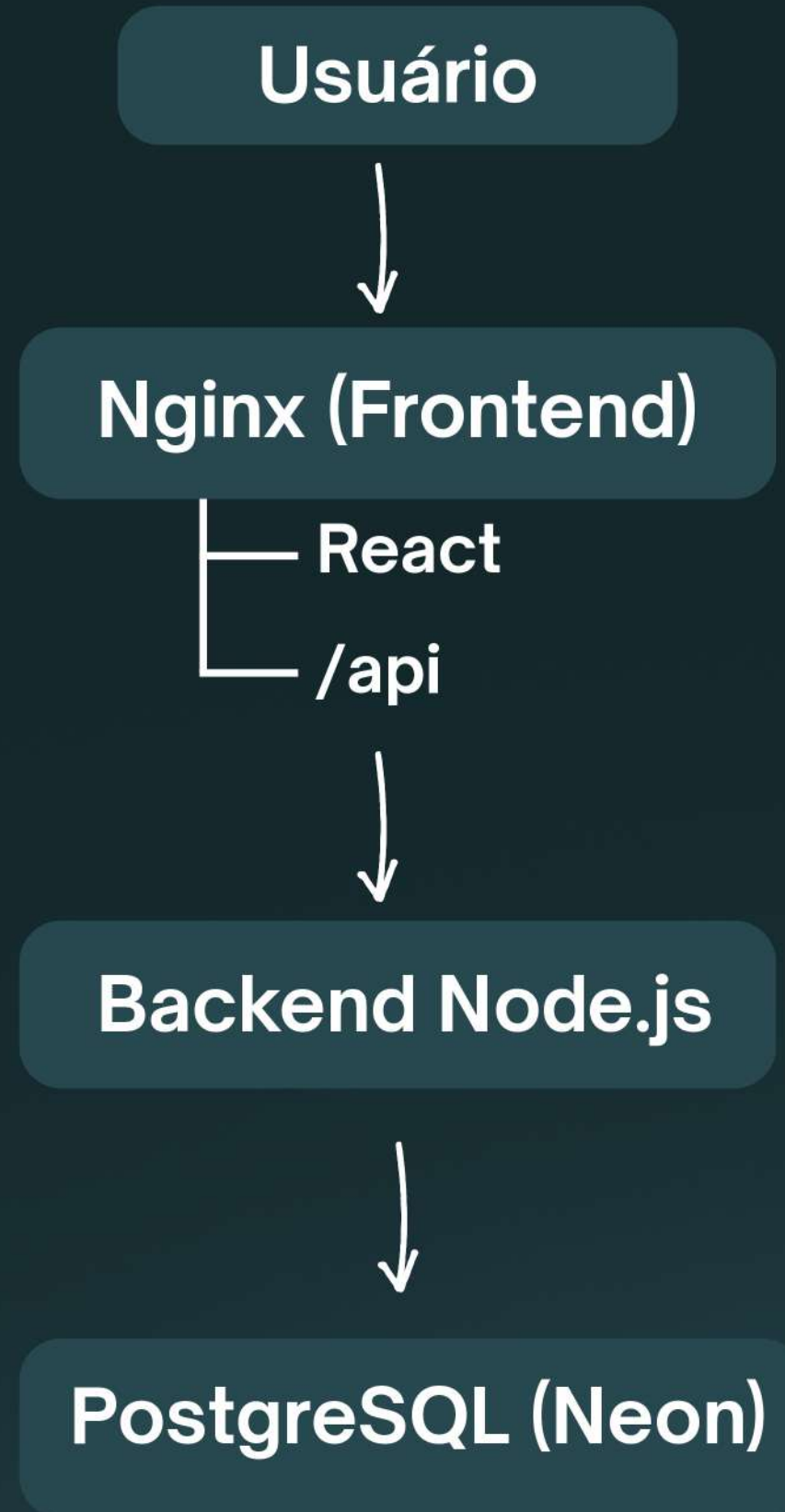
→ centralização em `validacoes.js`

Reorganização da estrutura do frontend

Infraestrutura e Implantação

Arquitetura de Implementação

2026



Objetivo:

- Ambiente de implantação reproduzível e padronizado
- Containerização da aplicação com Docker

Componentes:

- Nginx (Proxy Reverso)
- Frontend React
- Backend Node.js + Express
- Banco PostgreSQL (Neon)

Organização dos Containers

2026

Docker Compose

Container Frontend

↩ Nginx + React Build

Container Backend

↩ Node.js + Express



Banco de Dados
PostgreSQL na nuvem

O projeto foi dividido em dois containers principais. O container frontend contém o build da aplicação React servido pelo Nginx. O container backend contém a aplicação Node.js responsável pelas regras de negócio e acesso aos dados.

Ambos os containers são gerenciados por um único arquivo docker compose, que é responsável por criar a rede interna para comunicação e iniciar os serviços automaticamente.

O banco de dados foi mantido fora do ambiente Docker, hospedado no Neon, a fim de garantir a persistência dos dados e backups automáticos.

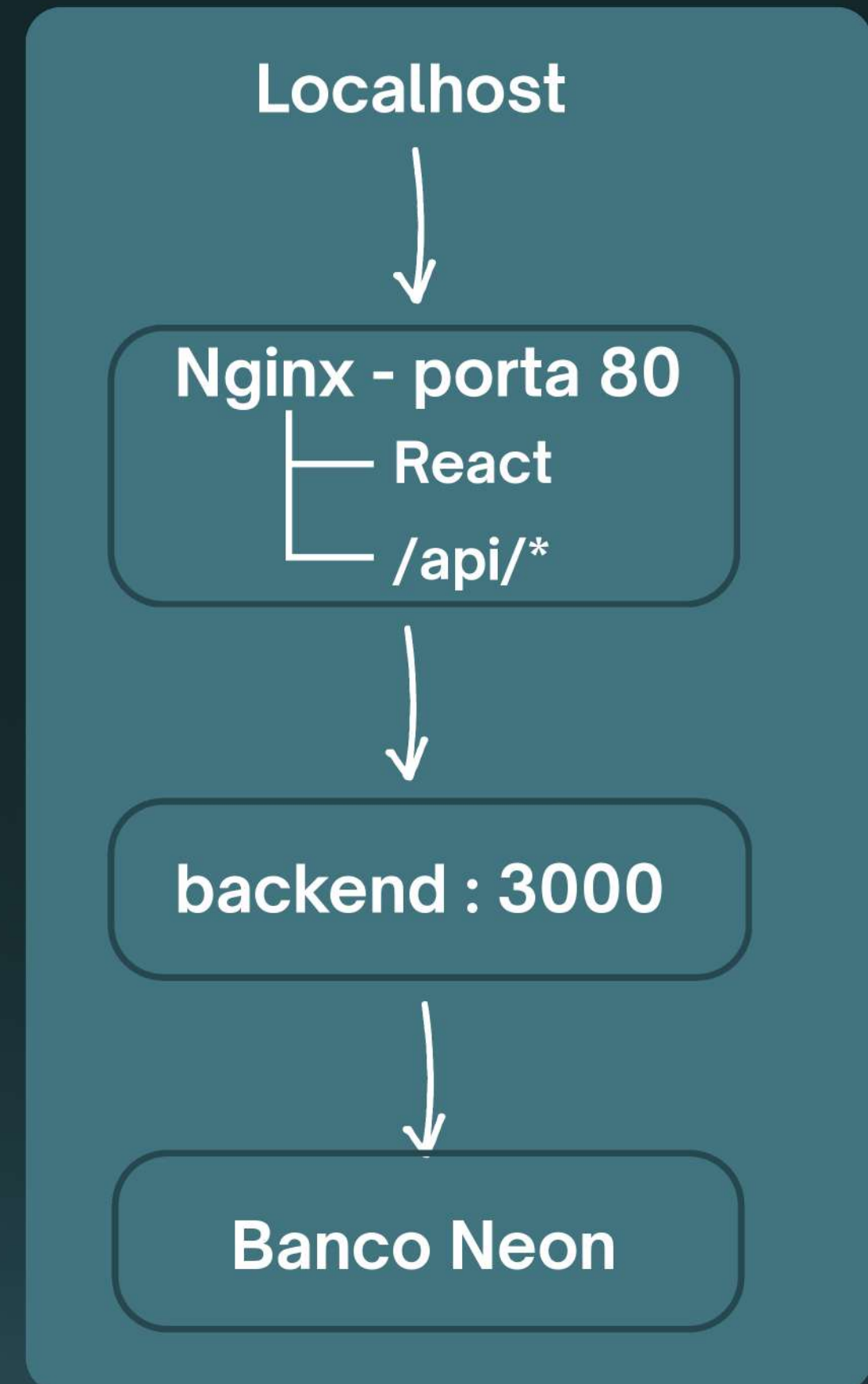
Comunicação entre serviços

2026

- Rede interna criada automaticamente pelo Docker Compose
- Nginx recebe requisições externas e encaminha chamadas da API para o backend
- Comunicação interna via hostname backend
- Menor exposição de portas e arquitetura mais próxima da produção

Conexão com o Banco de Dados

- Backend conecta ao PostgreSQL (Neon) pela internet
- Comunicação segura utilizando SSL
- Credenciais armazenadas em arquivo .env
- Configurações carregadas automaticamente na inicialização



Evidências

2026

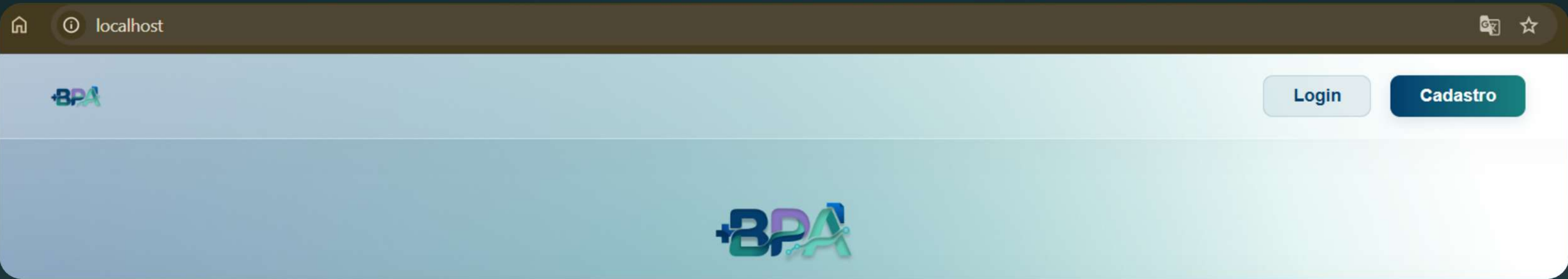
Imagem 1. Construção das imagens e iniciação dos containers.

```
✓ Image projeto-ds-backend Built
✓ Image projeto-ds-frontend Built
✓ Network projeto-ds_default Created
✓ Container bpa-backend Created
✓ Container bpa-frontend Created
```

Imagem 2. Containers em execução no Docker.

☐	▼	●	projeto-ds	-	-	-	0%	35 seconds ago				
☐		●	bpa-backend	93085c1031d0	projeto-ds-backend	3000:3000 ↗	0%	35 seconds ago				
☐		●	bpa-frontend	68325cca3891	projeto-ds-frontend	80:80 ↗	0%	35 seconds ago				

Imagem 3. Sistema em execução em <http://localhost>.



O que
construímos:

2026

Obrigada!

Equipe 2 - *Clntilante*

Ana Clara Bizarria <acsb2> Byanca Souza <bhcs> Jaianny Souza <jkos>
Layse Gomes <lmfg> Maria Claudia Rocha <mcsr> Samara Petrilly <spab>
Raiana Donato <rlpd>